# Despliegue

**Producción durante la competencia:** `generacion-solar-production-gtqero.laravel.cloud` en Laravel Cloud, región US East (Ohio), con Laravel MySQL 8.4 en configuración Dev y cómputo Flex con escala a cero. Cada `git push` a `main` despliega solo.

La aplicación necesita PHP 8.3 o superior, MySQL o PostgreSQL y un paso de compilación de assets con Node.

## Variables de entorno

```
APP_NAME="Generación Solar Guatemala"
APP_ENV=production
APP_KEY=            # php artisan key:generate --show
APP_DEBUG=false
APP_URL=https://tu-dominio
APP_LOCALE=es

DB_CONNECTION=mysql   # o pgsql
DB_HOST=...
DB_PORT=3306
DB_DATABASE=...
DB_USERNAME=...
DB_PASSWORD=...

ADMIN_EMAIL=admin@solar.gt
ADMIN_PASSWORD=       # cambiar antes de exponer la URL

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database
LOG_CHANNEL=stderr
```

Ningún secreto va al repositorio. `.env` está en `.gitignore` y `.env.example` documenta las claves sin valores.

## Comandos

**Compilación:**

```
composer install --no-dev --optimize-autoloader
npm ci && npm run build
```

**Arranque (cada despliegue):**

```
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

**Primera vez, para cargar los datos de demostración:**

```
php artisan db:seed --force
```

`db:seed` es idempotente: los seeders usan `updateOrCreate`, así que repetirlo no duplica registros. Para reconstruir todo desde cero: `php artisan migrate:fresh --seed --force`.

Si se cargan generaciones por fuera del formulario, hay que reconstruir las alertas:

```
php artisan solar:evaluar-alertas
```

## Laravel Cloud

1. Conectar el repositorio de GitHub.
2. Agregar un recurso de base de datos (Laravel MySQL 8.4, configuración Dev) vinculado al entorno; las variables `DB_*` se inyectan solas.
3. Build command: `composer install --no-dev --optimize-autoloader && npm ci && npm run build`
4. Deploy command: `php artisan migrate --force && php artisan config:cache && php artisan route:cache && php artisan view:cache`
5. Definir `APP_KEY`, `APP_URL`, `ADMIN_EMAIL` y `ADMIN_PASSWORD` en el panel de variables.
6. Tras el primer despliegue, ejecutar `php artisan db:seed --force` desde la consola del proveedor.

## Railway

Igual que lo anterior. Railway detecta Laravel con Nixpacks; hay que asegurarse de que la imagen incluya Node para el paso de assets y añadir el comando de arranque:

```
php artisan migrate --force && php -S 0.0.0.0:$PORT -t public
```

En producción real conviene php-fpm con Nginx en lugar del servidor embebido.

## Notas

- Con MySQL hay que evitar alias SQL que sean palabras reservadas (`real`, `order`, `key`). SQLite los tolera y el error solo aparece en producción; ya se corrigió uno en `EstadisticasService`.
- `AppServiceProvider` fuerza esquema HTTPS en producción, porque la aplicación corre detrás de un proxy TLS y de lo contrario los assets salen por `http` y el navegador los bloquea.
- El mapa consume mosaicos de Esri desde el navegador del visitante; no requiere clave ni configuración en el servidor, pero sí salida a internet desde el cliente.
- La API tiene límite de 60 peticiones por minuto por IP. Si se hace una demostración con muchas recargas seguidas, subir el límite en `AppServiceProvider`.

## Lista de verificación antes de entregar

- [ ] La URL pública carga el tablero con datos
- [ ] El mapa muestra los marcadores y el filtro por departamento responde
- [ ] `/admin` permite iniciar sesión y el escritorio muestra los indicadores
- [ ] Crear una generación por debajo del 80% genera la alerta automáticamente
- [ ] La ficha de una granja muestra la proyección y la precisión histórica
- [ ] `/docs/api` responde y `/api/v1/estadisticas/nacional` devuelve JSON
- [ ] `.env` no está en el repositorio y `.env.example` sí
- [ ] La contraseña del administrador no es la de ejemplo

---